

NTWIST

A General Problem-Solving Framework

This is just how to think about things getting done.

The idea is simple: define what you are trying to make true, agree on how you are going to approach it, break that approach into smaller whats, and repeat until the work is executable.

Core principle

The HOW at one level becomes the WHAT at the level below. Every handoff is an alignment conversation, not a game of telephone.

Version 1.0 | Internal use

FRAMEWORK

The framework on one page

A recursive way to move from intent to execution without losing the reason the work exists.

1

Destination

Where are we trying to go? Define the state of reality we actually want and the purpose behind it.

2

Artifact

What are we trying to build, change, decide, or enable that should move us toward that destination?

3

Problems

What is currently preventing that artifact or outcome from existing? Name the gaps rather than jumping straight to solutions.

4

Decomposition

What has to be done? Break the problem into smaller WHATs until the logic from input to desired output naturally holds and the next owner can engage with it.

5

Picture + causal check

If you could snap your fingers and have the result in front of you, what would it do? How would that solve the problem, and how would solving that problem move you toward the destination?

At every level

Lock in four things: PURPOSE / INTENT, WHAT, HOW, and VALIDATION. Then the agreed HOW is decomposed into the next level of WHATs.

The loop:

INTENT → WHAT ↔ HOW → DECOMPOSE → WHATs → REPEAT

Ask WHY upward. Ask HOW downward. Keep the WHAT at each boundary explicit.

The framework is fractal

The same conversation repeats from product intent all the way down to executable instructions.

A person does not simply receive a task and execute it. They receive a WHAT, understand the purpose behind it, challenge whether it is the right thing, agree on the HOW at their level, then decompose that HOW into smaller WHATs.

Product / Business	“Turn an engineering drawing into a proposed manufacturing routing.”	Defines the product outcome and why it matters.
Senior Architect	“Build six capabilities that together produce that routing.”	Chooses the high-level technical path, shaped by available technology, client needs, constraints, and maintainability.
Developer	“Extract explicit operation callouts from a drawing.”	Chooses the component-level method and breaks it into implementable subproblems.
Junior Developer	“Classify these text regions and return a structured operation object.”	Turns the subproblem into concrete logic and interfaces.
Computer	“Given this input, perform these exact operations and return this output.”	Executes the specification.

The key relationship

The senior architect’s HOW is the developer’s WHAT. The developer’s HOW is the junior developer’s WHAT. The junior developer’s HOW eventually becomes instructions precise enough for a computer.

Upward test: Why does this component exist, and how does completing it serve the parent problem?

Downward test: What smaller truths must become true for this component to work?

Every handoff is an alignment conversation

The point is to remove ambiguity before effort compounds it.

The assigning person should not throw a WHAT over the wall. They should discuss the HOW with the person doing the work - not to micromanage implementation, but to shape the vision together and keep the WHAT grounded in technical reality.

PURPOSE / INTENT	WHAT	HOW	VALIDATION
Why this matters and what parent problem it serves.	What must be true or produced at this level.	The agreed approach at this level - enough to guide the next breakdown.	The simple test that says this component is done.

A useful conversation sounds like this:

- “This is what I need to become true, and this is why it matters to the level above us.”
- “When you say that, do you mean X or Y? What is in scope? What does done look like?”
- “Given the technology, client context, constraints, and effort, I think the right high-level approach is $A \rightarrow B \rightarrow C$.”
- “Agreed. For A, B, and C, here is the validation test that tells us each component is complete.”
- “Good. You now own taking that agreed approach and having the same conversation one level down.”

A good handoff

The receiver understands the WHAT, understands the PURPOSE / INTENT, agrees with the HOW at the level being handed off, and knows the VALIDATION scheme. They can now reason independently without drifting away from the parent objective.

What should not happen

- The upper level prescribes detailed implementation without the person closest to the work shaping it.
- The lower level starts building before everyone agrees on what “done” means.
- A component is optimized locally even though nobody can explain how it helps the parent problem.
- A vague WHAT is passed downward and ambiguity simply gets pushed onto more junior people.

Decompose until the problem becomes understandable

The objective is not to solve everything at the planning level. It is to turn an ambiguous problem into smaller problems that can actually be solved.

The planner does not need to know the final algorithm. The planner needs to create WHATs to which the next owner can provide a HOW.

Practical stopping rule

You have decomposed far enough when the person you are talking to no longer looks at you like you just spoke a different language - and, more tangibly, when the components and their logic naturally explain how you get from the input you have to the output you want.

Example: engineering drawing → manufacturing routing

#	WHAT	Why the logic holds
1	Ingest the PDF	Input: engineering drawing. Output: drawing content available to the system.
2	Extract explicit operation callouts	Find the operations directly stated on the drawing.
3	Map callouts to shop operation names	Translate drawing terminology into the shop's controlled vocabulary.
4	Extract feature-dependent operations	Infer operations required by the geometry/features even when they are not explicitly called out.
5	Organize the operations	Combine explicit + inferred operations into a candidate sequence.
6	Modify to a final route	Apply shop-specific knowledge, constraints, inspections, and corrections to produce the proposed routing.

Notice what is intentionally missing: OCR choice, model selection, embeddings, rules engines, solvers, data structures, or implementation details. Those are HOW decisions for the owners of the decomposed problems.

Use the “snap your fingers” test

A solution is not well-defined just because the words sound right. Make it concrete enough that you can see what it does and explain why that matters.

Imagine you could snap your fingers and the thing existed in front of you. What would it actually do? What would you observe? What changes because it exists?

1. WHAT DO YOU SEE?	2. HOW DOES THAT SOLVE THE PROBLEM?	3. HOW DOES THAT MOVE THE GOAL?
Describe the artifact operating in concrete terms: its inputs, outputs, behavior, interaction, and visible result.	Name the mechanism. What burden, failure, uncertainty, bottleneck, or missing capability does the observed behavior remove?	Reconnect to the original destination. If the problem is solved, why are we actually closer to what we wanted?

Routing example

- What you see: upload a drawing and receive a proposed routing containing explicit operations, inferred feature-dependent operations, shop-standard names, and an ordered sequence ready for engineering review.
- How it solves the problem: the manufacturing engineer no longer has to construct the route from scratch; much of the interpretation and assembly work has already been done.
- How it moves the goal: routing creation becomes faster, more repeatable, and less dependent on scarce manual engineering effort, increasing the capacity of the manufacturing engineering function.

Final sanity check

If reality looked exactly like this, would we actually have what we wanted? If the answer is no, go back up the tree. You may have the wrong WHAT, the wrong HOW, or even the wrong destination.

Give every component a validation scheme

Validation should be simple enough to use and clear enough to stop arguments about whether a component is done.

A validation scheme is not a thesis. It is the smallest credible test that lets the team say: “Yes, that bit works; move on.”

Component	Example “done” test
PDF ingestion	Given the agreed drawing set, the system can open and expose the required page content without failure.
Explicit callout extraction	On the validation drawings, required operation callouts are captured at the agreed precision/recall threshold or accepted review standard.
Operation-name mapping	Known callouts map to the correct shop operation name; unknown/ambiguous cases are surfaced rather than silently guessed.
Feature-dependent operation extraction	The agreed feature test set produces the expected implied operations.
Sequencing	The candidate route respects agreed precedence and feasibility rules on the validation set.
Final route modification	A manufacturing engineer can review the proposed route and it meets the agreed acceptance threshold for usefulness/correctness.

Why this matters

Milestones are strongest when they represent increasingly credible states of the artifact, not merely completed activities. “Built a prototype that produces the expected curve” is more useful than “worked on simulator for two weeks.”

Validation also creates feedback

- If a component fails its validation test, fix or reframe that component before continuing.
- If several components pass but the whole artifact still does not solve the parent problem, move upward and challenge the decomposition or causal logic.
- If the validation itself cannot be stated clearly, the WHAT is probably still too vague.

Worked example: drawing to routing

The same framework at multiple levels of the organization.

Level	WHAT	HOW	VALIDATION
Product Manager	WHAT: Reduce manual routing construction by producing a proposed route from a drawing.	HOW: Automate the interpretation + construction steps enough that an engineer mainly reviews and corrects.	VALIDATE: A representative drawing produces a useful proposed route; review effort is materially lower than building from scratch.
Senior Architect	WHAT: Build a system that turns drawing content into a proposed routing.	HOW: Ingest → explicit ops → vocabulary mapping → implied ops → sequencing → shop-specific modification.	VALIDATE: Each capability has a clear interface and the end-to-end pipeline works on a validation set.
Developer	WHAT: Extract explicit operation callouts.	HOW: Choose the document/vision/text processing approach that best fits the drawings and downstream needs; decompose into detection, classification, normalization, and output.	VALIDATE: Agreed callouts are found reliably and returned in the expected structure.
Junior Developer	WHAT: Classify candidate text as an operation callout and normalize it.	HOW: Implement the agreed component logic, interface, tests, and error handling.	VALIDATE: Unit/integration tests pass against labelled examples and ambiguous cases are handled as designed.
Computer	WHAT: Execute the specified transformation on the given input.	HOW: Code / model inference / deterministic operations.	VALIDATE: Output matches the programmed/tested expectation.

Notice the ownership boundary

The upper level helps shape the HOW enough to keep it realistic. The lower level owns working through that HOW, discovering the detailed solution, and turning it into the next set of WHATs. This keeps autonomy without losing alignment.

The recursion stops only when interpretation is no longer the bottleneck.

- At high levels, the WHAT may be a business capability.
- At middle levels, it may be a service, module, or transformation.
- At low levels, it may be a function, test, prompt, data structure, or command.
- At the final boundary, the instruction is precise enough that the computer can execute it.

When work is stuck, debug the layer - not just the person

Most execution failures can be traced to a small number of broken links in the chain.

Failure mode	What it looks like	Response
Wrong / unknown WHAT	The team is building the wrong thing or the request is too vague to reason about.	Go upward. Reconfirm intent, picture the desired result, and rewrite the WHAT.
Unknown HOW	The WHAT is understood, but nobody knows a credible path.	Get domain knowledge, research alternatives, prototype, or bring in the right expertise.
Misalignment	Different people think the WHAT, HOW, or success standard means different things.	Re-run the alignment conversation and write down the agreed interpretation.
Unknown authority	The people doing the work cannot make required decisions or get approvals.	Clarify decision rights and escalation paths.
Unknown capacity	The logic is sound but time, people, access, data, or tools are insufficient.	Change scope, sequencing, resources, or timeline.
Unknown standard	Nobody can tell whether the component is done.	Add or repair the validation scheme.
Unknown control	The team cannot tell early enough that reality is diverging from the plan.	Create regular signal, intermediate milestones, and correction points.

Fast diagnostic order

1) Did we define the right thing? 2) Do we know enough to solve it? 3) Are we aligned? 4) Can decisions be made? 5) Do we have the capacity/access? 6) Can we tell whether each component is done? 7) Are we getting feedback early enough to correct course?

WORKSHEET

One-page problem-solving canvas

Use this at any level: company objective, product capability, architecture component, development task, or implementation detail.

1. PURPOSE / INTENT

What are we ultimately trying to accomplish at this level? Why does it matter to the parent problem?

2. WHAT

What exactly must become true, be built, be changed, or be produced?

3. SNAP-YOUR-FINGERS PICTURE

If it existed right now, what would you actually see it do? How would that solve the problem and move the larger goal?

4. PROBLEMS / GAPS

What currently prevents this WHAT from being true?

5. HOW - HIGH LEVEL

What is the agreed approach at this level? What are the major components or transformations?

6. DECOMPOSED WHATs

What smaller WHATs does this HOW create for the next level? Does the logic naturally carry the input to the desired output?

7. VALIDATION

For each component, what simple test tells us: "yes, that bit is done"?

8. ALIGNMENT CHECK

Does the receiver understand the WHAT, PURPOSE / INTENT, HOW, and VALIDATION? Can they now continue the conversation one level down?

CLOSING

Operating principles

The framework is intentionally lightweight. The discipline is in repeating it consistently.

1	Start with the thing you actually want. Do not let an intermediate deliverable quietly become the goal.
2	Keep purpose attached to every WHAT. A person should be able to explain what sits above their work and why their output matters.
3	Talk before you build. Alignment is cheaper than rework. Challenge the WHAT and shape the HOW before implementation compounds assumptions.
4	Split responsibility without creating walls. The assigning person helps shape the HOW so the WHAT stays realistic; the owner closest to the work carries the solution forward.
5	Decompose until the logic holds. The path from input to output should make sense naturally to the next competent owner.
6	Validate components, not effort. Prefer observable states and simple “done” tests over activity-based milestones.
7	Make the solution visible. If you cannot say what you would actually see it do, the idea is probably still too abstract.
8	Always reconnect downward work to upward purpose. The local solution is only good if it causally serves the parent problem.
9	When stuck, move up or down the tree. Move down if the WHAT is still too large. Move up if the local solution works but the larger problem remains unsolved.
10	Repeat until execution is obvious. At the bottom, the WHAT is precise enough for a computer. At the top, the destination is precise enough to know whether it was worth doing.

WHY upward. WHAT at the boundary. HOW downward. Validate as you go.

The point is not to make planning complicated. The point is to make ambiguity visible before it becomes expensive.

Internal source material consolidated into this framework:

Zine (desire / problem / solution and clarity of intent); Project Failure Diagnostic (definition, knowledge, alignment, resources, standards, feedback); Nuvoil Bed Life Extension Simulator milestone plan (progressive artifacts and explicit completion criteria).